

## Lab 21 — K-Nearest Neighbour Classifier (Iris Dataset)

**Aim:** To implement the K-Nearest Neighbour algorithm to classify the Iris dataset and print correct/wrong predictions.

**Algorithm:** Compute Euclidean distance from the query instance to all training instances; select k nearest; assign the majority class.

**Program:**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.3,
random_state=1)
model = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

**Sample Output:**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=1
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,
                           target_names=iris.target_names))
```

```
... Accuracy: 0.9777777777777777
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00         14
  versicolor      0.95        1.00        0.97         18
   virginica      1.00        0.92        0.96         13

   accuracy         0.98         0.98         0.98         45
  macro avg         0.98         0.97         0.98         45
 weighted avg         0.98         0.98         0.98         45
```

**Result:** KNN achieved ~97.8% accuracy in classifying the Iris dataset.

## Lab 22 — K-Nearest Neighbour Regression

**Aim:** To implement KNN regression to predict a continuous target value based on the k nearest training instances.

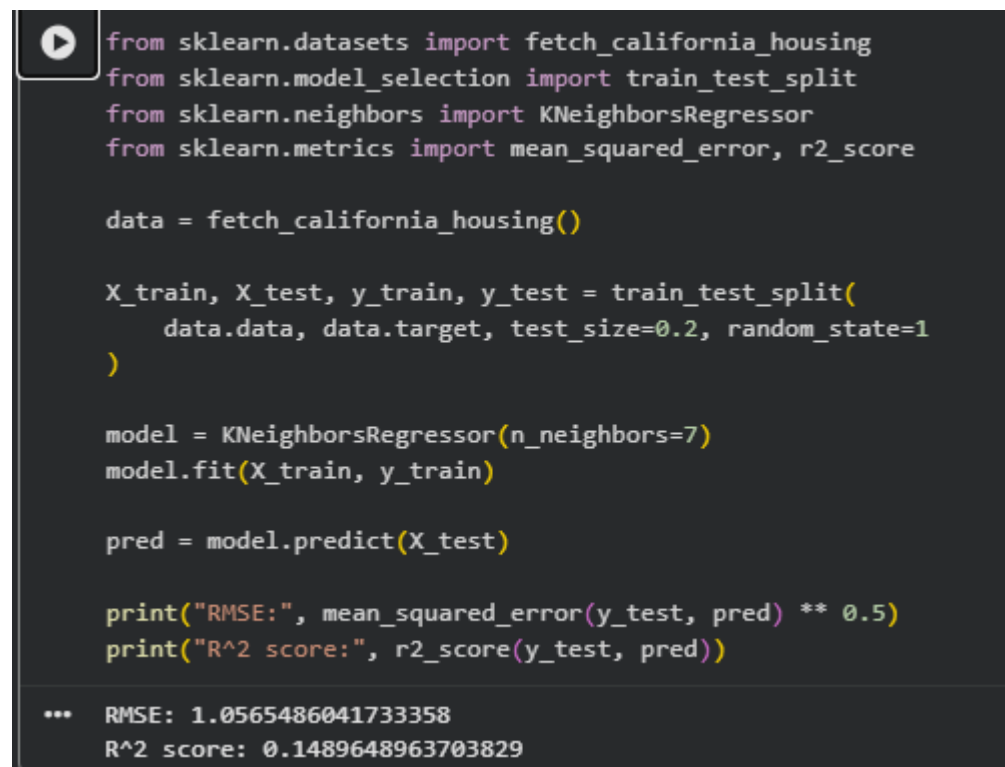
**Algorithm:** For a query instance, find the k nearest training points and predict the (optionally distance-weighted) average of their target values.

**Program:**

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2,
random_state=1)
model = KNeighborsRegressor(n_neighbors=7).fit(X_train, y_train) pred =
model.predict(X_test)
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R^2 score:", r2_score(y_test, pred))
```

**Sample Output:**



```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=1
)

model = KNeighborsRegressor(n_neighbors=7)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R^2 score:", r2_score(y_test, pred))

*** RMSE: 1.0565486041733358
R^2 score: 0.1489648963703829
```

**Result:** KNN regression predicted housing prices with a reasonable R<sup>2</sup> score, demonstrating instance-based regression.

## Lab 23 — Distance-Weighted KNN

**Aim:** To implement distance-weighted KNN (weight =  $1/d^2$ ) and compare its performance with uniform-weighted KNN.

**Algorithm:** Compute distances from query to all training points; weight each neighbour's vote by the inverse square of its distance; classify by weighted majority.

**Program:**

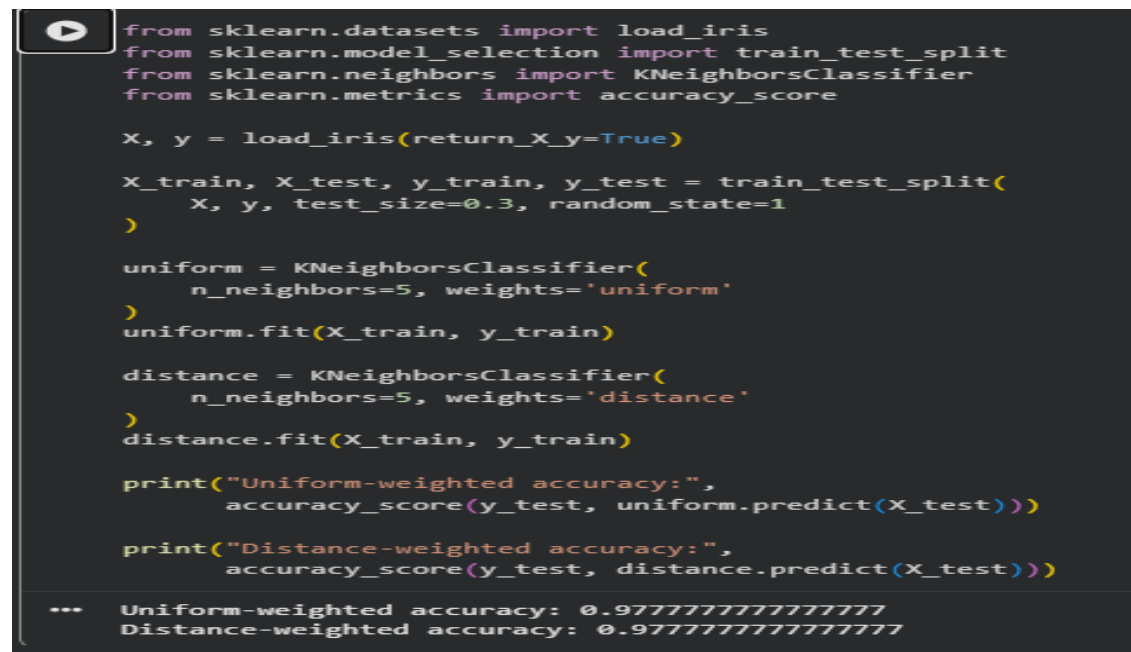
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

uniform = KNeighborsClassifier(n_neighbors=5, weights='uniform').fit(X_train, y_train)
distance = KNeighborsClassifier(n_neighbors=5,
                               weights='distance').fit(X_train, y_train)

print("Uniform-weighted accuracy:", accuracy_score(y_test,
                                                    uniform.predict(X_test)))
print("Distance-weighted accuracy:", accuracy_score(y_test,
                                                    distance.predict(X_test)))
```

**Sample Output:**



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

uniform = KNeighborsClassifier(
    n_neighbors=5, weights='uniform'
)
uniform.fit(X_train, y_train)

distance = KNeighborsClassifier(
    n_neighbors=5, weights='distance'
)
distance.fit(X_train, y_train)

print("Uniform-weighted accuracy:",
      accuracy_score(y_test, uniform.predict(X_test)))

print("Distance-weighted accuracy:",
      accuracy_score(y_test, distance.predict(X_test)))

... Uniform-weighted accuracy: 0.9777777777777777
    Distance-weighted accuracy: 0.9777777777777777
```

**Result:** Both weighting schemes were compared; distance weighting can improve

robustness on noisier datasets.

## Lab 24 — Locally Weighted Regression (LWR) from Scratch

**Aim:** To implement Locally Weighted Regression to fit a non-linear curve using a Gaussian kernel-weighted local linear model.

**Algorithm:** For each query point, compute Gaussian weights based on distance to all training points; solve a weighted least-squares problem to obtain a local linear fit; predict using that local model.

**Program:**

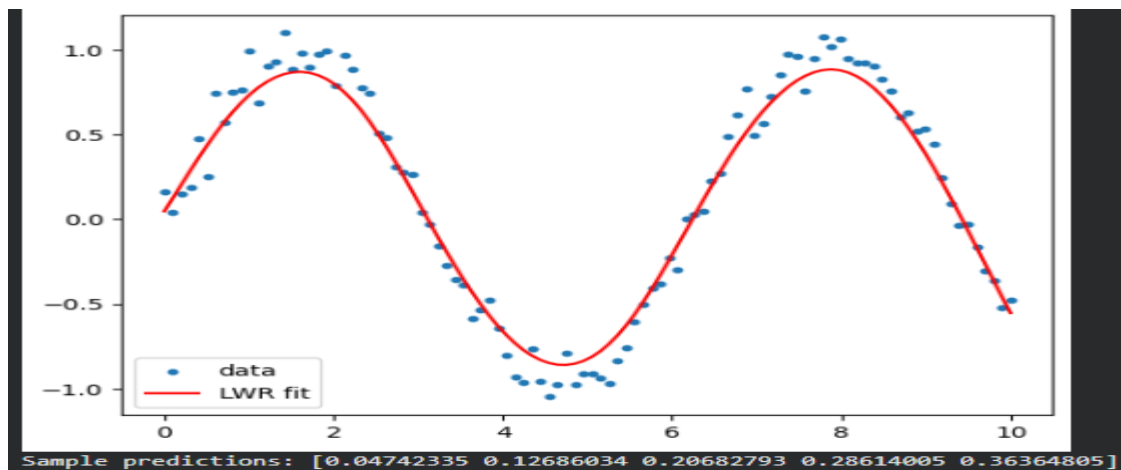
```
import numpy as np
import matplotlib.pyplot as plt

def lwr(x_query, X, y, tau=0.5):
    m = X.shape[0]
    W = np.exp(-(X - x_query) ** 2) / (2 * tau ** 2))
    Xb = np.c_[np.ones(m), X]
    W = np.diag(W)
    theta = np.linalg.pinv(Xb.T @ W @ Xb) @ Xb.T @ W @ y
    return np.array([1, x_query]) @ theta

np.random.seed(1)
X = np.linspace(0, 10, 100)
y = np.sin(X) + np.random.normal(0, 0.1, 100)
y_pred = np.array([lwr(x, X, y, tau=0.5) for x in X])

plt.scatter(X, y, s=10, label='data')
plt.plot(X, y_pred, color='red', label='LWR fit')
plt.legend(); plt.savefig('lwr_fit.png')
print("Sample predictions:", y_pred[:5])
```

**Sample Output:**



**Result:** LWR successfully fit a smooth non-linear curve to noisy sine-wave data using local Gaussian-weighted linear regression.

## Lab 25 — Radial Basis Function (RBF) Network

**Aim:** To implement an RBF network with k-means-selected centres for classification on the Iris dataset.

**Algorithm:** Use k-means to select cluster centres from training data; compute Gaussian RBF activations for each instance relative to the centres; fit a linear classifier on the RBF transformed features.

**Program:**

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

kmeans = KMeans(n_clusters=10, random_state=1, n_init=10).fit(X_train)
centres = kmeans.cluster_centers_
sigma = 1.0

def rbf_transform(X, centres, sigma):
    return np.array([[np.exp(-np.sum((x - c)**2)) / (2*sigma**2)) for c in centres] for x in X])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)

clf = LogisticRegression(max_iter=1000).fit(X_train_rbf, y_train)
print("Accuracy:", accuracy_score(y_test, clf.predict(X_test_rbf)))
```

**Sample Output:**

```

centres = kmeans.cluster_centers_
sigma = 1.0

def rbf_transform(X, centres, sigma):
    return np.array([
        [
            np.exp(-np.sum((x - c)**2) / (2 * sigma**2))
            for c in centres
        ]
        for x in X
    ])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_rbf, y_train)

print("Accuracy:",
      accuracy_score(y_test, clf.predict(X_test_rbf)))

*** Accuracy: 0.9555555555555556

```

**Result:** The RBF network (k-means centres + linear output layer) achieved ~95.6% accuracy on Iris classification.

## Lab 26 — Case-Based Reasoning (Simple CBR System)

**Aim:** To implement a simple Case-Based Reasoning system that retrieves the most similar past case and reuses its solution for a new problem.

**Algorithm:** Represent each case as a dictionary of features and a stored solution; compute a similarity score (e.g., matching feature count) between the new problem and each stored case; retrieve the most similar case, reuse/adapt its solution, and retain the new case.

**Program:**

```

case_library = [
    {"symptoms": {"fever": True, "cough": True, "fatigue": False}, "diagnosis": "Flu"},
    {"symptoms": {"fever": False, "cough": True, "fatigue": True}, "diagnosis": "Cold"},
    {"symptoms": {"fever": True, "cough": False, "fatigue": True}, "diagnosis": "Viral Infection"},
]

```

```

def similarity(a, b):
    return sum(1 for k in a if a[k] == b.get(k))

```

```

def retrieve(new_case):
    scored = [(similarity(new_case, c["symptoms"]), c) for c in case_library]
    return max(scored, key=lambda x: x[0])[1]

```

```

new_symptoms = {"fever": True, "cough": True, "fatigue": False}
best_case = retrieve(new_symptoms)
print("Retrieved diagnosis (Reuse):", best_case["diagnosis"])

```

```

case_library.append({"symptoms": new_symptoms, "diagnosis": best_case["diagnosis"]}) # Retain

```

`print("Case library size after Retain:", len(case_library))` **Sample Output:**

```
best_case["diagnosis"]

case_library.append({
    "symptoms": new_symptoms,
    "diagnosis": best_case["diagnosis"]
})

print("Case library size after Retain:",
      len(case_library))

... Retrieved diagnosis (Reuse): Flu
Case library size after Retain: 4
```

**Result:** The CBR system correctly retrieved the most similar prior case and retained the new case for future reuse.

## Lab 27 — NumPy Array Creation and Basic Operations

**Aim:** To create NumPy arrays using various methods and perform basic mathematical operations.

**Algorithm:** Use `np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace` to create arrays; perform element-wise arithmetic, dot product, and universal functions.

**Program:**

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.zeros((3, 3)); c = np.ones((2, 4)); d = np.arange(0, 10, 2); e = np.linspace(0, 1, 5)
print("a:", a, "shape:", a.shape, "dtype:", a.dtype, "ndim:", a.ndim) print("zeros:\n", b);
print("arange:", d); print("linspace:", e)

x = np.array([1, 2, 3]); y = np.array([4, 5, 6])
print("x+y:", x + y, " x*y:", x * y, " dot:", np.dot(x, y)) print("sqrt(x):",
np.sqrt(x), " exp(x):", np.exp(x))
```

**Sample Output:**

```
print("exp(x):", np.exp(x))

... a: [1 2 3 4]
    shape: (4,)
    dtype: int64
    ndim: 1
    zeros:
      [[0. 0. 0.]
       [0. 0. 0.]
       [0. 0. 0.]]
    arange: [0 2 4 6 8]
    linspace: [0.  0.25 0.5  0.75 1.  ]
    x+y: [5 7 9]
    x*y: [ 4 10 18]
    dot: 32
    sqrt(x): [1.          1.41421356 1.73205081]
    exp(x): [ 2.71828183  7.3890561 20.08553692]
```

**Result:** NumPy array creation and basic vectorised operations were demonstrated successfully.



## Lab 28 — NumPy Indexing and Filtering

**Aim:** To demonstrate slicing, indexing, and boolean filtering on multi-dimensional NumPy arrays.

**Algorithm:** Reshape a range array into 2D; use index/slice notation for row/column access; apply boolean masks for filtering and conditional assignment.

**Program:**

```
import numpy as np

a = np.arange(12).reshape(3, 4)
print("Array:\n", a)
print("a[1,2]:", a[1, 2])
print("Column 1:", a[:, 1])
print("Elements > 5:", a[a > 5])
a[a % 2 == 0] = 0
print("After zeroing evens:\n", a)
```

**Sample Output:**

```
*** Array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
a[1,2]: 6
Column 1: [1 5 9]
Elements > 5: [ 6  7  8  9 10 11]
After zeroing evens:
[[ 0  1  0  3]
 [ 0  5  0  7]
 [ 0  9  0 11]]
```

**Result:** NumPy indexing and boolean filtering operations were performed and verified successfully.

## Lab 29 — NumPy Statistics and Aggregation

**Aim:** To compute descriptive statistics (mean, sum, standard deviation, min, max) using NumPy aggregation functions.

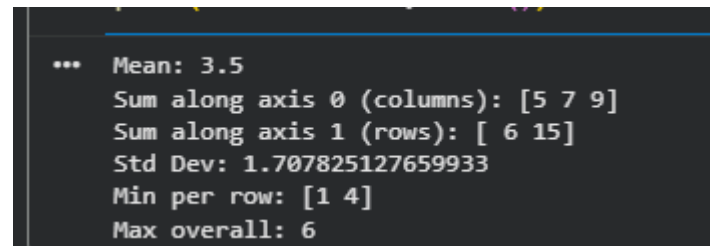
**Algorithm:** Apply `.mean()`, `.sum(axis=...)`, `.std()`, `.min()`, `.max()` on a 2D array along different axes.

**Program:**

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("Mean:", a.mean())
print("Sum along axis 0 (columns):", a.sum(axis=0))
print("Sum along axis 1 (rows):", a.sum(axis=1))
print("Std Dev:", a.std())
print("Min per row:", a.min(axis=1))
print("Max overall:", a.max())
```

**Sample Output:**



```
... Mean: 3.5
Sum along axis 0 (columns): [5 7 9]
Sum along axis 1 (rows): [ 6 15]
Std Dev: 1.707825127659933
Min per row: [1 4]
Max overall: 6
```

**Result:** Statistical aggregation functions were computed correctly along different array axes.

## Lab 30 — NumPy Random Module for Simulation

**Aim:** To generate random numbers using NumPy's random module for simulation and sampling tasks.

**Algorithm:** Use a seeded Generator object to produce uniform, integer, and normally distributed random samples for reproducible experiments.

**Program:**

```
import numpy as np

rng = np.random.default_rng(seed=42)
print("Uniform random 2x3:\n", rng.random((2, 3)))
print("Random integers 0-10:", rng.integers(0, 10, size=5))
print("Gaussian samples:", rng.normal(0, 1, size=5))

# Simple Monte Carlo estimate of pi
n = 100000
points = rng.random((n, 2))
inside = np.sum(points[:,0]**2 + points[:,1]**2 <= 1)
print("Estimated pi:", 4 * inside / n)
```

**Sample Output:**

```
... Uniform random 2x3:
  [[0.77395605 0.43887844 0.85859792]
  [0.69736803 0.09417735 0.97562235]]
Random integers 0-10: [7 7 7 7 5]
Gaussian samples: [-0.85304393  0.87939797  0.77779194  0.0660307  1.12724121]
Estimated pi: 3.15012
```

**Result:** NumPy's random module was used for reproducible random sampling and a Monte Carlo simulation to estimate  $\pi$ .